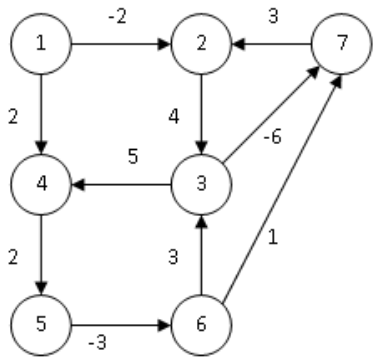
PROIECTAREA ALGORITMILOR
Examen – 05.06.2015
Timp de lucru: 1h30

SUBIECTUL 1 (11p = 4 x 2p + 1 x 3p) – 30 min



Se dă graful din imaginea alaturată.
1. Să se determine o sortare topologică a grafului, specificând clar pașii urmați pentru a ajunge la aceasta.
2. Pozitivând costul muchiilor negative prin aplicarea funcției modul, aplicați algoritmul lui Dijkstra, considerând sursa *s=1*. La fiecare pas, evidențiați structura cozii de priorități, precum și nodul extras din aceasta.
3. Considerând graful neorientat, aplicați algoritmul lui Tarjan pentru a identifica punctele de articulație din graf.
4. Pozitivând costul muchiilor negative prin aplicarea funcției modul, aplicați algoritmul lui Kruskal.
5. Aplicând primii pași ai algoritmului lui Johnson, calculați potențialul fiecarui nod și recalculați costurile muchiilor conform formulei folosite de algoritm (este suficient să recalculați costul pentru 2 muchii de cost pozitiv și două de cost negativ, la alegere).

SUBIECTUL 2 (20p = 5 x 4p) – 35 min

1. Care sunt pașii pe care trebuie să îi urmați pentru a rezolva o problemă prin programare dinamică? Exemplificați acești pași pentru problema parantezării matricilor studiată la curs, spunând înainte de toate cerința acestei probleme.
2. Care este numărul maxim, respectiv minim, de relaxări efectuate de către algoritmul Bellman-Ford pentru fiecare muchie a unui graf orientat? Dar pentru algoritmul lui Dijkstra?
3. Pornind de la definiție, explicați de ce este important ca euristica folosită de algoritmul A* să fie admisibilă? Demonstrați apoi că orice euristică consistentă este admisibilă.
4. Definiți ce înseamnă graful rezidual al unei rețele de transport și demonstrați (chiar și informal) că atunci când nu mai există nici o cale de ameliorare de la sursă la drenă în graful rezidual înseamnă că am descoperit tăietura minimală a rețelei de transport.
5. Explicați ce reprezintă o tăietura alfa-beta într-un arbore de joc și dați un exemplu de astfel de arbore, cu maxim 3 nivele, în care să scoateți în evidență minim două tăieturi alfa-beta.

SUBIECTUL 3 (9p) – 25 min

Intr-o pădure, toate cărările au aceeași lungime, iar Ursul tocmai trezit din hibernare vrea să ajungă de la bârlogul său până la râu cât mai repede întrucât îi este foame și sete. Însă Vulpea vrea să îi joace o festă și să îi blocheze Ursului toate căile cele mai scurte care îl duc la râu pentru a îl face să ocolească. Din păcate, pădurea este complicată și pot exista mai multe drumuri de aceeași lungime, iar Vulpea poate alege să blocheze doar o singură intersecție din pădure. Scopul vostru este să ajutați vulpea să decidă dacă există astfel de intersecții care, dacă ar fi blocate, garantează că ursul va merge o cale mai lungă decât în mod normal.

La intrare primiți *n* – număul de intersecții din pădure, *a[i][j]*, $1 \le i, j \le n$, matricea de adiacență a pădurii, *s* nodul/intersecția unde se află bârlogul Ursului și *t* nodul/intersecția unde Ursul preferă să se hrănească la râu. Ieșirea programului o reprezintă acele noduri care satifac cerința din enunț sau -1 dacă nu există nici un astfel de nod.

Sursa problemei: clasică, apare în mai multe cărți, forumuri, probleme de studiu, etc.

Pentru subiectul 3:
a. Explicați mai intâi care este ideea voastra de rezolvare (în cuvinte, încercați să oferiți cât mai multe detalii);
b. Spuneți ce structuri de date, tehnici de programare și/sau algoritmi clasici (prezentați la curs) veți folosi;
c. Schitați pseudocodul, fără a intra în detalii (inutile).
Atentie: Se dorește o complexitate cât mai bună! Determinați complexitatea soluției propuse!